Document number: P0593R0
Audience: WG21

Ville Voutilainen
2017-02-05

What to do with buffers that are not arrays, and undefined behavior thereof?

Abstract

There is a std-discussion thread at https://groups.google.com/a/isocpp.org/d/msg/std-discussion/rt2ivJnc4hg/Lr541AYgCQAJ that raises an interesting question; if C code creates a buffer with malloc, writes objects to it, and returns a pointer to that buffer, according to our object model rules, incrementing that pointer to treat the buffer as an array is undefined. Nothing in the program created an array. I can't find anything in the specification that suggests such a buffer is an array. An implementation probably wouldn't do anything funny, since it's unlikely that it can prove that the user didn't create an array, and fair amounts of existing code would probably break.

Another problem is that std::vector allocates an array of storage, but then treats that array as an array of element types. That's also undefined. This is captured by Core Issue 2182: http://www.open-std.org/jtc1/sc22/wg21/docs/cwg_active.html#2182.

What to do?

First of all, I don't know. I'm not suggesting that malloc should magically create objects. But creating a buffer with malloc and writing objects to the buffer doesn't create an array; therefore I don't see how to legally use such a buffer as an array.

There are various ideas suggested in the std-discussion thread mentioned in the abstract:

- Somehow allow such buffer uses to be valid.
- Add a new magic function that tells compilers and optimizers that the user promises that a pointer points to something that is an array.
- Add magical capabilities to the proposed span type.

This particular undefined behavior is probably not an actual problem in practice. But it's certainly weird that such UB exists in our specification, considering that such buffers appear in a lot of code that interfaces with C. It can even appear in code that is all C++, but yet our object model doesn't support it. The vector issue is all C++, and yet our object model doesn't support it.